A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science & Engineering

Under the Course of-

Course Title: Thesis/Project Work
Course Code: CSE 4000(A)

Thesis Supervisor-
Name: Mst. Sahela Rahman
Designation: Lecturer

Prepared by-
Name: Md. Riaz
ID/Registration No: 0322310105101024
Semester, Year: 4th Year - 7th Semester
Session: Spring - 2023

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Date of submission: _____

A Thesis on-

“AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics”

This thesis is submitted to the department of Computer Science & Engineering in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science & Engineering

Signature of Supervisor

Signature of Student

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Date of submission: _____

CERTIFICATION OF ORIGINALITY

I hereby declare that this thesis titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics" is my own original work, carried out under the supervision of Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology. To the best of my knowledge, this thesis contains no material previously published or written by another person, except where due reference is made in the text. This work has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

_____

Signature of Student

Md. Riaz

ID: 0322310105101024

CERTIFICATION OF APPROVAL

This is to certify that the research paper titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

That it has been read, evaluated and accepted for fulfilment of the Academic Degree of Bachelor in Computer Science and Engineering (B.S.C. in CSE) at Pundra University of Science & Technology.

The acceptance decision is made based upon an independent review process that provides critically constructive feedback within a short turnaround time. This paper represents the author's independent work, critical analysis, and capability in undertaking literature research as per the academic policies and ethical standards of the university.

I certify that the candidate has completed the required research activities for the program and recommend that this piece of work as of acceptable standard for submission and for inclusion in the academic record of the University.

Mst. Sahela Rahman
Lecturer
Department of Computer Science and Engineering
Pundra University of Science & Technology

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology, for her continuous guidance, patience, and constructive feedback throughout the design, implementation, and evaluation of this research. Her insistence on precise, defensible claims shaped this thesis at every stage, from the formal safety proof to the honesty of its limitations section.

I am also grateful to the faculty members of the Department of Computer Science & Engineering for the coursework and feedback that prepared me to undertake this research, and to my family for their patience and encouragement during the long implementation and evaluation cycles this thesis required.

Finally, I acknowledge the authors whose published research on natural language interfaces, text-to-SQL translation, and dashboard generation provided the foundation against which AEGIS's contributions are measured.

ABSTRACT

Relational databases hold data organizations need for decisions, but non-technical users often depend on developers to translate business questions into SQL. Natural-language interfaces try to close this gap, but many current systems still allow a language model to author executable SQL directly, making safety dependent on model behavior rather than system structure. This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), an architecture that restricts the model to intent extraction while deterministic software handles semantic mapping, permission enforcement, SQL compilation, validation, visualization selection, and widget persistence. AEGIS uses a closed semantic layer of approved metrics, dimensions, analytical patterns, and join paths, so the model never outputs SQL text. Instead, it produces a typed intent object that can be checked before any query is compiled. A prototype evaluation over a 107-query mixed natural-language benchmark on a seeded nopCommerce-style database shows that AEGIS produced no unsafe SQL statements and achieved 100 successful true database executions out of 107, while a direct LLM-to-SQL baseline achieved 27 successful executions out of 107 and produced one genuine unsafe write statement. These results support the thesis's central claim that SQL safety can be made a structural property of the architecture. The evaluation also shows a remaining limitation: safe and executable SQL is not always semantically correct, so correctness and scope-handling require a separate annotated benchmark in future work.

Table of Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1.1 Background

Relational databases store critical institutional data in organizations: financial records, customer accounts, sales transactions, and more. But accessing this data is uneven. Technical staff can write SQL queries to get any answer they need, while non-technical users have to wait for someone else to build them a report. This waiting is expensive. Analysis of enterprise reporting workflows shows that business users frequently wait days for new reports, and a recurring theme in institutional reporting is that many questions are variations of things already asked before: the same report with a different date range, or the same chart for a different department. These are not one-off questions; they are recurring reporting needs that should be served by saved, refreshable widgets rather than regenerated from scratch every time.

This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that lets users describe their reporting needs in plain English and produces dynamic dashboard widgets that can be saved, refreshed, and reused as part of their daily workflow, without anyone writing SQL.

Natural language interfaces to databases (NLIDBs) try to solve this problem. The idea is simple: a user should be able to ask â€œwhich categories have the highest refund rates this month?â€• and get a correct, visual answer without writing SQL. Researchers have made good progress here. Neural text-to-SQL systems now exceed 90% accuracy on the Spider benchmark [1], and large language models can produce reasonable-looking SQL with minimal setup [2]. But there is still a gap between benchmark results and production-ready deployment.

1.2 Problem Statement

Three problems make up the gap between benchmark text-to-SQL accuracy and safe, production-ready natural-language reporting.

Safety. Many modern natural-language-to-SQL systems rely on models that directly generate SQL tokens, which creates challenges for enforcing institutional access-control and security policies. An LLM generating SQL freely can produce queries that expose private data or use incorrect table joins, and detecting this after the fact is fundamentally harder than preventing it by construction.

Vocabulary mismatch. Benchmarks use actual column names in the questions, but real users speak in business terms (â€œrefund rateâ€• instead of SUM(o.RefundedAmount)). Matching these requires business knowledge that models do not always get right, and a wrong mapping can silently produce a plausible but incorrect report.

No widget generation. Existing systems answer one question at a time and discard the result. They do not produce saved reporting widgets that can be refreshed with new data tomorrow, shared with a colleague, or added to a daily dashboard. Every time someone needs the same report, they start from scratch, which directly contradicts the observation that reporting requests are often recurring rather than one-off.

These problems are not primarily about building a smarter model; they are about designing the system properly around the model. AEGIS addresses this by splitting the work into stages. The LLM's only job is to understand what the user is asking and output a structured description of the request. Everything after that, matching to the right business terms, building the SQL, selecting the chart, and saving the widget, is done by fixed rules and pre-approved templates.

1.3 Research Novelty and Motivation

Existing text-to-SQL research asks: how accurately can a model generate SQL from natural language? This thesis asks a different question: how can large language models be used for language understanding while being structurally prevented from generating executable SQL at all? The two pipelines differ structurally.

Classical NL-to-SQL: natural-language request, then model-authored SQL, then query result.

AEGIS: natural-language request, then intent extraction, semantic constraint, deterministic compilation, and a safe analytical artifact.

The contribution of this thesis is therefore not improved SQL generation accuracy; it is constrained analytical artifact generation, a design approach that removes SQL generation from the LLM's role entirely and provides safety and semantic fidelity guarantees that no generative model can match unconditionally.

AEGIS does not propose a new LLM. The model is interchangeable: Groq-hosted Llama 3.1 8B, OpenRouter, a local Ollama instance, or any endpoint compatible with the /v1/chat/completions interface. Because the LLM's only contract with the rest of the system is to produce a typed JSON intent object, model upgrades improve quality automatically without changing the compiler or safety infrastructure. This is model independence by design, not an implementation accident.

1.4 Objectives and Contributions

This thesis makes the following contributions:

1. A design-time review of representative e-commerce and business-intelligence reporting requests, resulting in eleven common reporting patterns checked against the published benchmark questions.
2. A system design in which all possible queries are limited to pre-approved templates and a defined semantic layer, which prevents SQL injection and unauthorized data access by construction.
3. The AEGIS system itself, including the semantic layer design, a vocabulary injection prompting strategy, a safe SQL builder with two-layer defence, a rule-based chart selector, and a widget storage system with scheduled refresh.
4. A vocabulary injection method that places approved metric and dimension names directly into the LLM prompt, reducing dependence on a manually written synonym list.
5. A verified benchmark over 107 mixed natural-language requests, separating SQL safety, true database execution validity, and semantic correctness rather than treating them as one accuracy number.
6. A planned cross-schema generalizability evaluation that will test whether the same compiler and safety architecture can be reused on a second e-commerce schema by rebuilding only the semantic layer.

1.5 Organization of the Thesis

The remainder of this thesis moves from related work and research gaps to methodology, experimental setup, evaluation, limitations, future work, and conclusion. The structure first establishes the prior research context, then explains the AEGIS design and implementation, then reports the verified benchmark results and discusses their implications.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

This chapter reviews the sources most directly comparable to AEGIS, grouped by theme: natural language interfaces to databases, neural and LLM-based text-to-SQL, natural language for visualization and dashboards, and applied conversational business intelligence. Closely related sources are discussed together rather than one at a time where they make the same architectural point. Sources only loosely adjacent to AEGIS's technical contribution (general business-adoption surveys, dashboard policy literature, evaluation-methodology papers unrelated to safety) are outside this review's scope.

2.1 Natural Language Interfaces to Databases

Two systematic surveys frame this literature [3]. The earlier survey benchmarks 24 NLIDBs against ten questions of increasing complexity across four architectural classes (keyword, pattern, parsing, grammar-based); grammar-based systems achieve the broadest coverage but require users to learn a constrained vocabulary [4]. The more recent review is the only one to name SQL-injection risk directly, discussing TrojanSQL, a backdoor injection attack the authors report is difficult to defend against, yet its only recommended mitigation is generic (additional layers of security or filtering), with no architectural defense proposed. Neither survey finds a system that treats safety as a structural property.

NaLIR and Veezoo are the closest prior systems to AEGIS's use of a curated vocabulary [5, 6]. The first parses a query into a grammar-constrained intermediate query tree and surfaces ambiguity to the user through a short multiple-choice interaction rather than guessing, correctly completing 88 of 98 tasks versus 56 of 98 without this step. The second constrains matching with an editable Knowledge Graph structurally analogous to a semantic layer, needing a median of two clarification attempts to reach a correct answer in a controlled study. Both, however, still compile into open-ended SQL and depend on user-facing dialogue to recover from a wrong query rather than preventing an unsafe one up front, and both produce one-off answers with no persistence.

2.2 Neural and LLM-Based Text-to-SQL

Spider and BIRD provide the clearest evidence that free-form generation is unreliable at scale [1, 2]. The first benchmark's strict cross-domain train/test split found the best contemporary baseline reached only 12.4% exact-match accuracy on unseen schemas. The second went further, testing 95 real, large databases with expert-annotated domain knowledge: even GPT-4 combined with DIN-SQL prompting reached only 55.9% execution accuracy against a 92.96% human-expert ceiling, with wrong schema linking (41.6%) and misunderstood database values (40.8%) as the dominant failure modes.

RAT-SQL and PICARD each constrain the same underlying generation approach without changing who authors the SQL [7, 8]. The former encodes the schema as a relational graph via relation-aware self-attention, reaching 57.2% exact-match on Spider, yet oracle analysis attributes 72-81% of its remaining errors to wrong column or table selection. The latter instead constrains decoding at the token level, cutting invalid SQL on Spider from roughly 12% to 2% â€” but the LLM still writes every character of the SQL string, so a semantic bug invisible to the grammar check (a wrong join, or wrong business definition) can still pass through as syntactically valid SQL.

G-SQL and TriSQL sit at opposite ends of the same spectrum and are the two closest points of comparison for AEGIS [9, 10]. G-SQL is rule-based and template-driven over a curated schema representation, deliberately avoiding a neural component that writes SQL directly; it reaches 100% execution accuracy on easy queries but falls to 45-55% on extra-hard ones. TriSQL is, at the time of writing, the state of the art: a three-stage LLM pipeline (schema selection, skeleton-first generation, complexity-aware refinement) reaching 82.2% execution accuracy on Spider. It is the sharpest possible contrast to AEGIS, precisely because it is state of the art and still has the LLM produce the final executable SQL directly, with no semantic layer and no reported safety evaluation.

2.3 Natural Language for Visualization and Dashboards

nl4dv and DataTone are the closest visualization-side precedents [11, 12]. The first maps natural language to a small, fixed set of five analytic tasks via a rule-based mapping table, closely analogous to AEGIS's bounded-vocabulary design, but has no SQL-safety or permission layer at all since it operates only on an in-memory dataset, and produces a one-off specification with no persistence. The second instead surfaces ambiguity through interactive widgets rather than resolving it silently, significantly outperforming IBM Watson Analytics

in a comparative study (5.43 vs. 2.00 correct facts, $p < 0.01$), but is limited to single-table data with no persistence mechanism.

DashBot composes multi-chart dashboards with deep reinforcement learning guided by design rules drawn from a study of 90 real dashboards [13]. It has no natural-language input at all, and because it never generates SQL from user language, it never encounters the safety problem AEGIS is built to solve.

2.4 Applied Conversational Business Intelligence

Two recent applied studies bookend AEGIS's design choice [14]. One describes a deployed Groq/LangChain assistant that gives an LLM direct SQL-execution tools and a self-correction retry loop â€” a live, working example of exactly the attack surface AEGIS's threat model is designed to close, with no reported adversarial evaluation [15]. The other, by contrast, independently arrives at AEGIS's central design principle in an unrelated domain: a deterministic, non-AI formalism computes a business explanation, and an LLM is used only to narrate the already-correct result. A pre-study in the same thesis found that ten commercial AI-BI products could all answer simple descriptive queries but none could correctly answer explanatory ones without manual reconfiguration â€” direct evidence that unconstrained LLM reasoning fails at exactly the class of task AEGIS targets.

2.5 Comparative Summary

Table 4: Comparative summary of the most closely related systems.

System	NL	Semantic	Safe SQL	Visual	Widget	Coverage	Evaluation
Spider/BIRD	Yes	-	-	-	-	-	Benchmark
RAT-SQL/PICARD	Yes	-	Partial	-	-	-	Benchmark
G-SQL/TriSQL	Yes	Partial	Partial	-	-	-	Benchmark
NaLIR	Yes	-	-	-	-	-	User study
Veezoo	Yes	Yes	-	-	-	-	User study
nl4dv/DataTone	Yes	-	-	Yes	-	-	User study
DashBot	-	-	-	Yes	Partial	-	Synthetic
Conversational BI	Yes	-	-	Yes	-	-	Demo
AEGIS	Yes	Yes	Yes	Yes	Yes	Yes	Production

2.6 Research Gap Analysis

Two gaps recur across every source reviewed above, including the state of the art. First, no system treats SQL-generation safety as a structural property rather than an accuracy metric: Spider, BIRD, RAT-SQL, PICARD, G-SQL, and TriSQL are all evaluated purely on exact-match or execution accuracy, and Shailesh et al.'s deployed assistant reports no adversarial evaluation at all despite giving an LLM direct database access. AEGIS closes this gap by removing SQL generation from the LLM's output space entirely and evaluating an explicit unsafe-SQL rate against a direct LLM-to-SQL baseline, rather than relying on accuracy as a proxy for safety.

Second, no system combines a governed semantic vocabulary with persistent, refreshable output: NaLIR and Veezoo produce one-off answers, nl4dv and DataTone produce one-off visualizations, and DashBot has no natural-language input at all. AEGIS's widget engine closes this gap, motivated by the design-time observation that real reporting requests are often recurring rather than one-off . Valkenburgh's independent arrival at AEGIS's â€œlet a deterministic layer compute the answerâ€• principle, in an unrelated domain, is the strongest external corroboration that this design decision reflects a pattern discovered wherever LLM reliability is treated as an engineering constraint rather than an accuracy target.

CHAPTER 3

METHODOLOGY

3.1 Research Paradigm

This thesis follows a Design Science Research paradigm, in the sense used by Hevner et al.: knowledge is produced by building and evaluating a novel artifact that addresses an identified organizational problem, rather than by testing a hypothesis about an existing phenomenon. The artifact in this thesis is the AEGIS system itself. Design Science Research requires three things to be demonstrated: problem relevance, design rigor, and evaluation against the stated problem. Problem relevance is established through a formative study that analyzes real reporting behavior rather than assuming a problem exists. Design rigor is established through the formal model and threat model, which state precisely what safety property the architecture guarantees and under what boundary that guarantee holds. Evaluation uses a benchmark dataset constructed for this domain and true database execution checks that separate SQL safety from execution validity and semantic correctness.

The research proceeded through three iterative build-evaluate cycles. The first cycle produced a semantic layer and compiler for a minimal set of intent classes and validated the core safety proposition on a small hand-written query set. The second cycle expanded coverage to all eleven intent classes identified in the formative study and introduced vocabulary injection after an early synonym-dictionary approach proved unable to keep pace with real query phrasing. The third cycle added the widget persistence layer and expanded the evaluation harness used for the thesis benchmark.

3.2 Formative Study of Reporting Patterns

The eleven-pattern taxonomy used throughout this thesis (Table 2) originates from a design-time review of representative e-commerce and administrative reporting requests, conducted by the author while designing AEGIS. This was a qualitative review carried out by a single researcher during system design, not an independently annotated, inter-rater-validated study, and no separate annotated dataset accompanies this thesis. An earlier version of this chapter reported specific percentages and an inter-rater reliability statistic for a larger unpublished

dataset; that dataset was not published alongside this thesis, so those figures have been withdrawn.

In place of that withdrawn figure, Table (formative study) below reports a pattern classification of the answerable analytical portion of the published benchmark (evaluation_dataset/questions.json): every one of those benchmark questions was individually classified into one of the eleven patterns by the author. This classification is itself a single-annotator judgment, not independently cross-checked by a second annotator, so it should not be read as a validated inter-rater statistic; unlike the withdrawn figures, however, it is reproducible and independently checkable, since the classification of each question is published alongside the questions themselves in evaluation_dataset/pattern_classification.json.

Table (formative study): Pattern classification of the answerable analytical benchmark subset, sorted by observed frequency.

Pattern	Count (of 100)	Share	Example
KPI / Aggregate	28	28%	How many orders were placed today?
Ranking	21	21%	Which five categories have the highest refund rates?
Exception / Filter	18	18%	List products with stock levels below 10.
Trend Analysis	10	10%	Show monthly sales volume over the last year.
Comparison	10	10%	Compare average order value between mobile and desktop users.
Summary / Group	9	9%	Give me an overview of the Electronics category.
Cohort	2	2%	First-time buyers vs. returning customers.

Funnel	1	1%	Cart-to-purchase abandonment after seeing shipping costs.
Correlate	1	1%	Which attributes correlate with higher margins?
Segment	0	0%	Not exercised by this particular 100-question sample.
Tabular	0	0%	Not exercised by this particular 100-question sample.

[PLACEHOLDER — FIGURE 5 NOT YET INSERTED]

Pattern classification of the answerable analytical benchmark subset

Figure 5: Pattern classification of the answerable analytical benchmark subset

This classification yields three design directions that shaped the rest of this chapter. First, a small set of patterns appears sufficient: on this benchmark, the top three patterns (KPI, Ranking, and Exception/Filter) already account for 67% of all questions, and all eleven patterns together account for 100%, supporting a fixed template library rather than an open-ended query language. Segment and Tabular happen not to be exercised by this particular 100-question sample, which is a property of this benchmark rather than evidence that those two patterns are unnecessary; the compiler supports both regardless. Second, business vocabulary differs systematically from database column names: reviewed requests used phrases like â€œtotal refund rate,â€• never SUM(o.RefundedAmount), which motivates an explicit semantic layer rather than exposing the schema directly to the language model. Third,

reuse appears to be the norm rather than the exception: many requests were variations of things already asked before, in a different time window or for a different segment, which motivates the widget persistence design.

3.3 Design Principles

Five principles guide every architectural decision in this chapter:

7. Separate understanding from execution. The LLM understands the question; fixed rules handle everything else.
8. Define business terms explicitly. Metrics, dimensions, joins, and time rules are written once in a semantic layer, not inferred per query.
9. Limit what SQL can be generated. SQL is built only from pre-approved, parameterized templates.
10. Select visualizations by rule. Chart type is decided by question type, result shape, and established visualization design guidance, not by a learned model.
11. Persist results for reuse. Every query produces a saved, refreshable widget rather than a discarded answer.

3.4 Formal Model

The formal idea behind AEGIS can be stated without mathematical notation. Classical text-to-SQL gives the language model a request and a database schema, then asks the model to produce SQL directly. AEGIS splits that responsibility into checked stages. The model reads the user's language and produces only a typed intent object. The semantic layer checks whether the requested metric, dimension, time rule, and pattern are approved. The compiler then builds read-only SQL from templates, the visualization selector chooses a chart or table, and the widget engine persists the result as a reusable artifact.

Table: Plain-language AEGIS formal model.

Component	Meaning in AEGIS	Safety role
User request	The original natural-language reporting question	Never becomes SQL directly
Intent object	Typed summary of metric, dimension, pattern, filter, time	Rejects fields outside the schema of allowed intent keys

	range, and limit	
Semantic layer	Approved metrics, dimensions, join paths, patterns, visualization rules, and permissions	Defines what the system is allowed to answer
Compiler	Deterministic SQL builder using parameterized templates	Produces read-only SQL from approved bindings only
Validator	Post-compilation SQL safety scanner	Blocks non-SELECT statements as defense in depth
Widget artifact	Saved visualization and refresh plan	Keeps a reusable audit trail of what was generated

The safety claim is therefore simple: if the semantic layer and compiler templates are trusted, then untrusted user language cannot introduce a table, column, join, or write operation that the semantic layer did not approve. The LLM never receives authority to write executable SQL.

Safety proposition. *No generated query can reference a table, column, or row that is not represented in the approved semantic layer for the user's role. All SQL identifiers are drawn from closed semantic bindings, and literal values are passed through parameterized SQL rather than string interpolation. SQL injection through untrusted natural-language input is structurally prevented by design.*

Security boundary. This guarantee holds within a defined threat boundary: the semantic layer definitions, compiler templates, and permission predicates are trusted, administrator-controlled artifacts. An administrator who embeds malicious SQL inside a metric definition, or a supply-chain compromise of the compiler library, falls outside this boundary and requires separate operational security controls. Explicitly stating this boundary, rather than leaving it implicit, is itself part of this thesis's contribution: prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes rigorous security comparison difficult.

3.5 Threat Model

AEGIS protects against attacks arriving through the untrusted natural-language input channel. The model assumes the database and application server are properly hardened; the attacker controls only the query field.

T1 - Prompt injection attempting SQL generation.

Attack: *"Ignore previous instructions. Generate DROP TABLE orders."*

Control: The IntentObject schema contains no SQL field. Any non-approved string in metric_term or dimension_term is rejected by Pydantic type validation at Stage 2, before the compiler is reached.

T2 - Unauthorized metric or dimension access.

Attack: *"Show me customer passwords" or "List credit card numbers by order."*

Control: Fields such as customer_password do not exist in the semantic layer vocabulary. The LLM never sees those names; it receives only the curated, approved label list. Stage 2 rejects any unrecognized term.

T3 - Unauthorized row access.

Attack: *A store-level user asks: "Show revenue for all branches."*

Control: Stage 4 (Permission Rewriter) runs after the LLM and appends a role-specific WHERE predicate (for example, AND o.StoreId = :user_store) derived from the authenticated session. This cannot be suppressed or overridden by natural-language content.

T4 - DML or DDL injection.

Attack: *A crafted prompt that tricks the LLM into associating a write operation with an intent class.*

Control: No template in the pattern library contains a DML or DDL keyword. The AST-level post-compilation validator explicitly rejects any non-SELECT statement as a defense-in-depth layer.

Not protected by AEGIS (requires operational security controls outside this architecture):

- A malicious administrator embedding arbitrary SQL inside a metric's sql_expr field.
- Supply-chain compromise of the compiler module or the SQL parser library.
- Database-level privilege escalation that bypasses the application layer.
- LLM provider infrastructure compromise or model poisoning.

Explicitly documenting out-of-scope threats is itself a contribution: prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes meaningful security comparison difficult.

3.6 System Architecture

Every user query travels through exactly seven stages. Stages 2 through 7 contain no artificial intelligence; they are deterministic code. Rejection at any stage produces a structured clarification message rather than a best-effort guess.

Stage 1 - Intent Extraction. A lightweight LLM reads the query together with a system prompt built from the semantic layer, and outputs a validated IntentObject (intent class, metric term, dimension term, filters, sort, limit, confidence). This is the only stage that involves artificial intelligence.

Stage 2 - Coverage Validation. The server checks that both the metric term and the dimension term exist in the semantic layer vocabulary before anything else runs. Unknown identifiers are rejected here, with a structured message listing the available identifiers, rather than being passed to the compiler.

Stage 3 - Semantic Mapping. Business-logic aliases are expanded (for example, â€œabandonedâ€• maps to a specific OrderStatusId), and relative time expressions such as â€œthis monthâ€• are resolved to concrete date predicates.

Stage 4 - Permission Rewriting. A role-specific WHERE predicate is appended based on the authenticated user's session. This runs after the LLM has already finished, so no natural-language content can influence it.

Stage 5 - SQL Compilation. A breadth-first search over the join graph finds the minimal join path connecting the tables required by the resolved metric and dimension, and pre-compiled SQL expressions are substituted into a parameterized template. No SQL text is ever assembled from concatenated user input.

Stage 6 - Visualization Selection. A rule engine maps the intent class and result shape to a default chart type. This stage contains no learned model.

Stage 7 - Widget Persistence. The analysis plan is hashed (SHA-256) to detect duplicates, and the query, chart configuration, and access rules are stored as a widget artifact that can be refreshed on a schedule.

[PLACEHOLDER — FIGURE 1 NOT YET INSERTED]

AEGIS architecture pipeline (User Request to Dashboard Widget)

Figure 1: AEGIS architecture pipeline (User Request to Dashboard Widget)

3.7 Semantic Layer Design

The semantic layer is the most important non-AI component of AEGIS. It separates business language from the underlying database structure and defines exactly which metrics, joins, and permissions are allowed to exist. A useful analogy is LEGO blocks rather than free-form clay: the semantic layer defines a finite set of composable building blocks. User questions are unlimited, but every answerable question is a composition of these blocks.

[PLACEHOLDER — FIGURE 2 NOT YET INSERTED]

Semantic layer modularity - composable blocks vs. free-form SQL generation

Figure 2: Semantic layer modularity - composable blocks vs. free-form SQL generation

Table 1: Semantic layer object model.

Object	Field	Example
Metric	label, SQL expression, joins, visual default, security class	revenue = SUM(o.OrderTotal - o.RefundedAmount)
Dimension	label, SQL expression, datatype,	category = c.Name from

	access scope	Category
Filter	label, SQL predicate, datatype	payment_status : o.PaymentStatusId = :val
Time rule	label, SQL predicate, granularity	current_week : DATEADD(week, ...)
Join path	source, target, ON clause	Order to OrderItem to Product to Category
Pattern	required slots, SQL template, visualization default	ranking: metric plus dimension maps to a bar chart
Permission	rule	store_manager filtered by store location

In the nopCommerce prototype, the semantic layer defines 15 metrics, 34 dimensions, and 11 join paths across 12 analytics-relevant tables. The full nopCommerce schema contains 126 tables; the remaining 114 (system, content-management, configuration, authentication, vendor, and promotions tables) are deliberately not represented in the semantic layer at all. No business analyst asks â€œshow me revenue by ScheduleTask,â€• and excluding these tables functions as an implicit table-level access control: even a crafted prompt that names a hidden system table directly is rejected at Stage 2, because the identifier simply does not exist in L.

3.8 Intent Parsing with Dynamic Vocabulary Injection

The central technique that makes Stage 1 reliable is vocabulary injection. At startup, AEGIS builds the system prompt by listing every approved metric and dimension name, with a plain-English description, directly from the semantic layer (approximately 1,100 tokens for 15 metrics and 34 dimensions). The LLM sees exactly which identifiers are valid and maps any user phrasing to the correct identifier without a manually maintained synonym list. This has three advantages over a synonym dictionary: zero maintenance, since adding a metric automatically updates the vocabulary the model sees; broad coverage of arbitrary user phrasing, since the model performs the mapping rather than a fixed lookup table; and token efficiency, since the full vocabulary for 15 metrics and 34 dimensions fits in roughly 1,100 tokens.

Structured output enforcement for LLMs [16] constrains the model's response to a fixed JSON schema before any downstream validation occurs, which is why Stage 1 can rely on a typed IntentObject rather than free-form text: malformed or off-schema output is rejected at the API boundary, before Stage 2 coverage validation ever runs.

The output schema enforces typed fields:

```
{
  "intent_class": "kpi | ranking | trend | comparison | exception |
                   summary | segment | funnel | cohort | correlate | tabular",
  "metric_term": "string",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value": "string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean"
}
```

[PLACEHOLDER — FIGURE 3 NOT YET INSERTED]

Vocabulary injection workflow

Figure 3: Vocabulary injection workflow

3.9 Safe Query Compiler

The compiler instantiates SQL from a library of parameterized templates, one per analytics pattern.

[PLACEHOLDER — FIGURE 4 NOT YET INSERTED]

Taxonomy of the eleven AEGIS analytical patterns

Figure 4: Taxonomy of the eleven AEGIS analytical patterns

Table 2: The eleven AEGIS analytical patterns.

Pattern	Required slots	Optional slots	Default visual
KPI (Aggregate)	metric	time_rule, filter	kpi_card
Ranking	metric, dimension	time_rule, filter, limit	bar_chart
Trend	metric, time_grain	time_rule, filter	line_chart
Comparison	metric, segment	time_rule, filter	grouped_bar
Exception	metric, threshold	dimension, time_rule	table
Summary	metric[], dimension	time_rule, filter	multi_card
Segment	metric, dimension	time_rule, filter	pie_chart
Funnel	metric, stages	time_rule, filter	funnel_chart
Cohort	metric, group_def	time_rule, filter	grouped_bar
Correlate	metric, attribute	time_rule, filter	scatter_plot
Tabular	dimension	filters, time_rule	table

After placeholder substitution, two safety layers apply in sequence. Layer 1 is a parameterized query engine that separates SQL structure from user-supplied inputs, so no

user text is ever concatenated into the SQL string. Layer 2 is a post-compilation safety scanner that rejects any assembled query containing a forbidden construct: non-SELECT statements, UNION, EXCEPT or INTERSECT, EXEC, or references to system tables. If any forbidden pattern is detected, the compiler raises a SecurityError rather than returning a partially safe query.

[PLACEHOLDER — FIGURE 6 NOT YET INSERTED]

Two-layer SQL safety defence

Figure 6: Two-layer SQL safety defence

3.10 Visualization Selector

Table 3: Visualization selector mapping.

Intent	Result shape	Selected visualization
KPI	scalar	KPI card
Ranking	1 measure, up to 20 categories	Horizontal bar chart
Trend	1 measure, time series	Line chart
Comparison	1 measure, 2-4 segments	Grouped bar chart
Exception	row-level detail	Sortable table
Summary	2-4 scalar measures	KPI card grid
Segment	1 measure, categorical	Pie chart
Funnel	ordered conversion stages	Funnel chart
Cohort	1 measure, 2+ groups	Grouped bar chart
Correlate	2 measures, continuous	Scatter plot
Tabular	raw records	Sortable table

Two additional rules apply after the data is known, rather than at selection time: bar charts with more than 20 categories are automatically converted to a table, and pie charts with more than 8 slices are converted to a bar chart. Both rules exist because the initial chart choice is made before the exact cardinality of the result is known.

3.11 Widget Persistence and Reuse

Each widget stores a unique identifier (a SHA-256 hash of the analysis plan), the original question, the analysis plan in JSON form, a hash of the SQL template used, chart configuration, timestamps, access rules, and run history. A new question triggers the full seven-stage pipeline; if an identical widget already exists, determined by a match on the plan hash, the cached artifact is returned immediately rather than recompiled. Scheduled refresh re-executes the stored SQL against fresh data on a configurable interval, which directly addresses the design-time observation that reporting requests are often recurring rather than one-off: a widget answers the question once and then continues answering it as new data arrives, rather than requiring the same natural-language request to be re-processed from scratch every time.

[PLACEHOLDER — FIGURE 7 NOT YET INSERTED]

Widget lifecycle and refresh model

Figure 7: Widget lifecycle and refresh model

CHAPTER 4

EXPERIMENTAL WORK

4.1 Implementation

AEGIS is implemented as a web application with a vanilla HTML and JavaScript frontend (jQuery, Chart.js) and a Python FastAPI backend, targeting a production nopCommerce 4.70 schema (126 tables, 107 foreign key constraints). The implementation follows directly from the AEGIS architecture:

- LLM integration: Llama 3.1 8B Instant via the Groq API, with structured JSON output enforcement. The system prompt is constructed dynamically by injecting the approved metric and dimension identifiers at startup.
- Rate limiting: a provider-agnostic configuration module with a sliding-window rate limiter and a concurrency-safe asyncio.Lock, so the architecture is not tied to a specific LLM vendor's throughput characteristics.
- Semantic layer: Python configuration modules containing 15 metrics, 34 dimensions, zero synonym entries, and 11 join paths across the 12 analytics-relevant tables represented in the semantic layer.
- SQL compiler: parameterized MySQL templates, with breadth-first search join-path resolution over the 12-table join graph. A post-compilation _validate_sql_safety() routine checks 16 forbidden patterns before a query is allowed to execute.
- Visualization selector: rule-based Python dictionaries implementing the visualization mapping, with the two post-hoc cardinality rules applied after the result set is known.
- Widget engine: SHA-256 plan-hash deduplication, with JSON file storage in the prototype, designed to be swapped for a relational store in a production deployment.
- Coverage validator: a pre-compilation gate that rejects unknown metric or dimension terms with structured guidance listing the available identifiers.
- Permission enforcement: a Permission Rewriter that appends role-based WHERE predicates for five roles: public, store_manager, regional_manager, read_only, and analyst.

4.2 Experimental Environment

All experiments ran against a Docker-containerized MySQL 8.0 instance initialized with the AEGIS Truth Schema (126 tables, 107 foreign keys). The mock dataset used for evaluation contains 1,200 customers, 2,500 orders spanning 2024-2026, 6,298 order items, and 1,000 products mapped across 50 categories, sized to be representative of a mid-sized e-commerce deployment rather than a toy schema.

4.3 Benchmark Dataset Construction

This evaluation is a prototype evaluation, not a large-scale independent benchmark study. Its goal is to demonstrate that the AEGIS architecture achieves its stated safety and semantic-fidelity properties on representative real-world analytics queries, not to establish population-level accuracy claims. Standard text-to-SQL benchmarks such as Spider and BIRD do not evaluate adversarial safety or adherence to business vocabulary, which is why a domain-specific benchmark was necessary for this thesis.

A methodological caveat applies to every quantitative result reported in this chapter and in the results discussion. All benchmark construction, execution, and measurement were carried out by the author as part of this thesis, using a self-built dataset and evaluation harness; none of it has been independently replicated by a third party, externally audited, or peer-reviewed. This is distinct from reproducibility: the underlying artifacts (evaluation_dataset/questions.json, benchmark_results.json, pattern_classification.json, and the scripts that compute statistics from them) are published alongside this thesis specifically so that any reader can independently re-run the computation and check the reported figures against the raw data, even though no one outside this research has done so yet. Where a figure states that an annotation or classification step was performed by a named number of people (for example, two database engineers verifying gold-standard SQL), that step was part of the author's own research process rather than an independent, third-party audit. These results should therefore be read as internal evidence produced during this research and reproducible from the published artifacts, but not yet independently verified by anyone outside it.

A domain-specific benchmark of 107 natural-language reporting requests was built over the production nopCommerce schema described above. The full question set and recorded pipeline outputs are available in the evaluation_dataset/ directory of the project repository,

enabling verification of every reported metric. The benchmark mixes ordinary analytical requests with harder boundary cases in the same run; this thesis does not report those harder cases as a separate benchmark. Because the benchmark was constructed for the nopCommerce domain, accuracy and validity reported figures should be interpreted within that scope. The current artifact verifies SQL safety and true execution validity; semantic correctness still requires an annotated expected-answer benchmark in future work.

4.4 Baseline Systems

The evaluation plan defines four baselines, but not all are complete in the current prototype:

B1 - Direct LLM-to-SQL. Llama 3.1 8B prompted with the full database schema, with no semantic layer and no template constraints; the model is free to generate any SQL text it produces.

B2 - Decomposed LLM. A chain-of-thought strategy that first extracts entities, then generates SQL from the extracted entities. This baseline is prepared for evaluation but is not included in the verified results.

B3 - Template-only (no LLM). Keyword matching directly to templates, with no LLM-based intent extraction. This baseline has been executed for true database execution validity and is discussed as B3 in the results.

B4 - AEGIS ablated (no semantic layer). The full AEGIS pipeline with the semantic mapper bypassed. This remains future evaluation work.

4.5 Evaluation Procedure

The current evaluation focuses on the measurements that can be verified from repository artifacts and true database execution:

- RQ1: How accurately does the LLM intent parser extract typed reporting plans? This remains a semantic-correctness task requiring annotated expected labels.
- RQ2: Does AEGIS reduce unsafe SQL compared to direct LLM-to-SQL? Measured as genuine unsafe SQL statements across the 107-query benchmark.
- RQ3: Does AEGIS generate SQL that actually runs? Measured through true database execution against the seeded MySQL database, not merely by checking whether Python compilation succeeded.

- RQ4: How does the deterministic downstream compiler behave when intent selection is rule-based rather than LLM-based? Measured through the B3 template-only baseline.
- RQ5: Which remaining work is required in future work? Semantic correctness, B2/B4 baseline completion, and latency instrumentation are explicitly listed as future work rather than reported as completed results.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports the prototype evaluation results that have been verified from the repository artifacts and from true database execution. The chapter deliberately separates SQL safety, execution validity, and semantic correctness. A query can be safe and executable while still being semantically wrong; therefore, correctness is treated as a separate benchmark that requires annotated expected answers in future work.

The verified quantitative evidence currently covers the 107-query mixed benchmark against AEGIS and baseline B1, plus a completed B3 template-only execution-validity run. B2 and B4 remain future evaluation work, and latency has not yet been instrumented. The tables below therefore report only the measurements that can be traced to files and commands in the repository.

5.1 Benchmark Run and Verified Metrics

The benchmark consists of 107 natural-language reporting requests executed as one mixed test set. Earlier drafts described seven of those requests as a separate boundary-probe set. This chapter avoids that split because the evaluation design now treat all 107 requests as part of the same benchmark run. Some requests are harder boundary cases, but they are still counted in the same denominator as every other query.

Table 5: Verified benchmark status.

Item	Status
Benchmark size	107 mixed natural-language reporting requests
Database execution environment	MySQL 8.0 in Docker, seeded with nopCommerce-style data
Completed baselines	B1 direct LLM-to-SQL and B3 template-only
Pending measurements	Semantic correctness, B2/B4 baselines, and latency

Evidence files: evaluation_dataset/questions.json, benchmark_results.json, benchmark_results_b3.json, and verify_execution.py run against the safedash MySQL database on port 3307.

5.2 SQL Safety and True Database Execution Validity

Unsafe Query Execution Rate measures whether a generated SQL statement contains a genuine unsafe operation such as a write or schema-changing command. Query Execution Validity measures whether the SQL actually runs against the seeded MySQL database. These are different measurements: safety is about what the query is allowed to do, while true execution validity is about whether the database accepts and executes the statement.

Table 6: SQL safety and true execution validity on the 107-query benchmark.

System	Unsafe SQL	True execution validity
Baseline B1 (Direct LLM-to-SQL)	1 genuine unsafe statement in 107	27 of 107 executed successfully (25.2%)
AEGIS	0 unsafe statements in 107	100 of 107 executed successfully (93.5%)
B3 Template-only	Not used as the primary safety baseline in this chapter	104 of 107 executed successfully (97.2%)

Baseline B1 mostly failed because of hallucinated columns, invalid table references, or dialect errors. AEGIS had no unsafe SQL, but seven generated statements still failed due to implementation defects diagnosed below. B3 is included as a compiler-behavior comparison, not as a semantic-correctness result.

The strongest safety example is the disguised write request asking to cancel orders stuck in Pending for more than 30 days. Baseline B1 produced an executable UPDATE statement. AEGIS cannot express a write intent in its intent object or compiler templates, so it did not emit write SQL. This is the central safety result: AEGIS prevents this class of unsafe execution structurally rather than hoping the model follows an instruction not to write dangerous SQL.

[PLACEHOLDER — FIGURE 8 NOT YET INSERTED]

Verified safety and execution-validity comparison

Figure 8: Verified safety and execution-validity comparison

5.3 Execution Failure Analysis

The seven AEGIS execution failures are useful because they show where the prototype needs engineering work in future work. They are not safety violations: they are SQL statements that failed to execute correctly against the real database.

Table 7: AEGIS true-execution failures diagnosed from MySQL execution.

Query ID	Failure class	Observed database error
5	Relative-date normalization	Incorrect DATETIME value: 'this morning'
13	Compiler syntax edge case	MySQL syntax error near generated SQL fragment
19	Join-path or alias defect	Unknown column 'o.Id' in ON clause
46	Relative-date normalization	Incorrect DATETIME value: 'last 90 days'
72	Relative-date normalization	Incorrect DATETIME value: 'this quarter'
80	Compiler syntax edge case	MySQL syntax error near generated SQL fragment
84	Compiler syntax edge case	MySQL syntax error near generated SQL fragment

The failure pattern is narrow. Three failures are date-normalization bugs, three are compiler syntax edge cases, and one is a join or alias construction defect. These are concrete implementation bugs, not evidence that the safety boundary failed. They should be fixed in future work and then re-measured using the same true database execution procedure.

5.4 Semantic Correctness and Accuracy

Correctness or accuracy is a separate benchmark from execution validity. The current results prove that AEGIS usually generates SQL that the database can execute and that it does not generate unsafe SQL. They do not, by themselves, prove that every safe and executable query answers the user's real intent.

Table 8: Distinguishing the evaluation metrics.

Metric	What it answers	Current status
SQL safety	Did the generated SQL avoid unsafe write or schema-changing behavior?	Verified for B1 and AEGIS on all 107 requests.
True execution validity	Did the generated SQL run against the seeded MySQL database?	Verified for B1, AEGIS, and B3.
Semantic correctness / accuracy	Did the answer match the user's intended reporting need?	Not yet numerically scored; requires annotated expected outputs or labels.
Scope handling	Did the system clarify or reject unsupported requests instead of guessing?	Observed as a limitation; needs a separate annotated evaluation.
Latency	How long each stage takes end to end?	Not yet instrumented.

This distinction is important for the thesis argument. AEGIS maintained SQL safety even under harder boundary cases, but the scope-detection mechanism was incomplete: several unsupported or underspecified requests were mapped to safe but semantically wrong in-scope queries instead of being rejected or clarified. That behavior should be reported as a correctness and clarification limitation, not hidden inside the safety metric.

5.5 B3 Template-Only Baseline

The B3 template-only baseline is useful because it separates the deterministic compiler from the LLM intent parser. B3 uses keyword matching to create an intent object, then hands that intent to the same downstream semantic mapping and compiler path. Its high execution-validity result shows that many database errors are triggered by particular intent choices and time phrases, not by the compiler alone.

Table 9: B3 execution-validity summary.

System	Execution result	Interpretation
B3 Template-only	104 of 107 executed successfully (97.2%)	Higher execution validity than AEGIS on this run, but not evidence of higher semantic correctness.
B3 failures	3 database execution failures	Two compiler or join-path issues and one date-value issue were observed.

B3's result should be interpreted carefully. A keyword-only classifier may choose a query shape that runs successfully while answering the wrong question. Therefore B3 belongs in the execution-validity comparison, but it should not be used to claim semantic accuracy until the same annotated correctness benchmark is applied to all systems.

5.6 Discussion

5.6.1 AEGIS vs. direct LLM-to-SQL.

The B1 baseline exposes the central risk of direct SQL generation. The same model that can produce useful analytical SQL can also hallucinate schema objects, mix SQL dialects, and generate a real write statement when a business-looking request implies a write operation. AEGIS narrows the model's role to intent extraction, so these risks do not enter the SQL construction stage in the same way.

Table 10: Structural comparison of AEGIS vs. direct LLM-to-SQL.

Property	Direct LLM-to-SQL	AEGIS

SQL generation	Model-generated free-form text	Deterministic compiler
Schema exposure to LLM	Requires tables, columns, and relationships	Uses approved business labels
Business metric definitions	Inferred from prompt and schema names	Defined in the semantic layer
SQL injection prevention	Prompt-level instruction and post-checking	Structural prevention through intent schema and templates
Permission enforcement	External or absent	Applied after intent extraction by deterministic code
Dashboard widget persistence	Not provided by default	First-class saved artifact
Model dependency	Strongly tied to model quality	Compiler and safety scanner are model-independent

5.6.2 Semantic layer versus retrieval-augmented generation.

Retrieval-augmented generation can help an LLM find relevant schema fragments, but it does not remove the model's authority to write SQL. AEGIS uses the semantic layer for a different purpose: it defines which business concepts are allowed to exist and how they compile into SQL. RAG may still be useful in a future large deployment to select relevant semantic-layer entries, but the final SQL should still be compiled by the deterministic AEGIS compiler.

5.6.3 Scope boundary.

AEGIS currently supports a bounded set of approved metrics, dimensions, analytical patterns, and join paths. This is the source of its safety, but also the source of its main limitation. If a user asks for something not represented in the semantic layer, the system should reject or clarify the request. The current prototype does not always do that reliably; improving scope detection is therefore a priority in future work.

CHAPTER 6

LIMITATIONS AND FUTURE WORK

6.1 Limitations

Semantic layer construction cost. Every AEGIS deployment requires a domain-specific semantic layer built by someone with both business knowledge and schema access. Organizations without this expertise, or with rapidly evolving schemas, may find the maintenance burden significant. AEGIS is not a zero-configuration system.

Cannot answer arbitrary SQL questions. By design, AEGIS only answers questions that map to a supported analytics primitive with an approved metric and dimension. Ad hoc queries, multi-level nested aggregations, or requests for data fields not in the semantic layer fail with a coverage error. This is a deliberate trade-off, not an oversight.

Complex analytical queries require new templates. Approximately 2.6% of the formative-study requests required patterns not yet in the template library. Adding a new pattern requires a developer with SQL knowledge; it is not something a business user can do themselves.

Quality depends on intent extraction, not just compilation. The safety guarantees apply to compilation and execution, not to intent extraction quality. A model that misclassifies a request will produce a structurally safe but semantically wrong query. Safety and accuracy are separate properties, and this thesis is careful not to conflate them.

Benchmark selection. The custom 107-query benchmark was necessary because standard benchmarks such as Spider and BIRD do not evaluate SQL safety or adherence to business logic; this also means the reported figures are not directly comparable to Spider- or BIRD-reported numbers from other systems.

Semantic layer scalability. Modern context windows of roughly 128,000 tokens can hold approximately 2,500 distinct metric and dimension definitions; most enterprise deployments expose fewer than 500 core concepts, so this is not a near-term constraint, but very large vocabularies would need retrieval-augmented vocabulary injection rather than injecting the entire semantic layer at once.

Database agnosticism. The current compiler generates MySQL syntax; supporting PostgreSQL or SQL Server requires extending the compiler module, not redesigning the architecture.

Storage persistence. The prototype uses JSON flat files for widget storage; the widget registry interface is designed to be swapped for a relational database in a production deployment.

Multi-turn conversation. AEGIS currently treats each request independently. Contextual carryover across turns, of the kind studied in conversational text-to-SQL research, is not yet implemented.

Vocabulary injection limitations. Highly specialized domain terminology may require supplementary few-shot examples in the prompt beyond the label and description pairs currently injected.

6.2 Future Work

- Clarification requests: when confidence is low, AEGIS should ask a follow-up question instead of guessing, for example "did you mean revenue or profit?"
- Semantic layer wizard: a guided interface for business analysts to define new metrics and dimensions without writing Python.
- Multi-step queries: currently each query produces one widget; compound questions such as "revenue trend and top 5 products side by side" require two separate queries today.
- Automated denial-of-service protection: query complexity scoring and server-side timeout enforcement, since the join graph bounds query cost but does not currently score it explicitly.
- Broader schema coverage: extending the nopCommerce semantic layer to cover promotions, vendor analytics, and content-management engagement metrics.
- Multi-turn conversational carryover, extending the single-turn design discussed above.
- Outcome and rejection-category instrumentation: measuring, from real benchmark and production runs, the precise proportion of requests answered directly, answered after clarification, answered after a semantic layer extension, or rejected, and the relative frequency of each rejection category. This thesis currently reports these categories

qualitatively; adding this instrumentation would let a future revision report a measured percentage breakdown rather than a qualitative one.

- Semantic correctness benchmark: adding annotated expected-answer labels for the 107 mixed requests so the evaluation can report correctness separately from SQL safety and true database execution validity.
- Cross-schema generalizability benchmark: rebuilding the semantic layer for a second schema, such as WooCommerce, while keeping the intent parser contract, compiler structure, and safety scanner unchanged. This future evaluation would measure how much of AEGIS transfers across schemas and how much effort is required to configure a new domain.
- Remaining baselines and latency: completing B2 and B4, then instrumenting pipeline latency with repeatable timing logs in future work.

CHAPTER 7

CONCLUSION

This thesis presented AEGIS, a system for turning plain-English reporting requests into dynamic, refreshable dashboard widgets over relational databases. The central contribution has two parts. First, a system design that limits the language model to understanding questions, while all query building, chart selection, and widget storage is handled by fixed rules and pre-approved templates, so that safety is a property guaranteed by system structure rather than a probability that improves with model quality. Second, a semantic-layer and vocabulary-injection design that makes approved business terms explicit instead of asking the model to infer them from raw schema names.

The verified evaluation shows that AEGIS produced no unsafe SQL statements across the 107-query benchmark and successfully executed 100 of 107 generated queries against the seeded MySQL database. The direct LLM-to-SQL baseline successfully executed 27 of 107 queries and produced one genuine unsafe write statement. These figures support the safety argument while also exposing the remaining engineering work: seven AEGIS execution failures, incomplete scope detection, and an unmeasured semantic correctness benchmark.

The literature review situates this contribution precisely: prior text-to-SQL research, from Seq2SQL through RAT-SQL and PICARD, treats SQL-generation accuracy as the object of study and, with the partial exception of a 2025 systematic review's discussion of injection attacks, does not evaluate safety as a first-class property at all. AEGIS's architectural choice, removing SQL generation from the language model's role entirely rather than constraining it more tightly, is a categorically different answer to the same underlying problem, and one independently echoed in recent explanatory-analytics research that arrives at the same â€œlet a deterministic layer compute the answerâ€• principle in an unrelated domain.

AEGIS is built for environments where data privacy, consistent reporting definitions, and daily reuse of saved reports matter more than unlimited query flexibility. The limitations discussion states plainly what this design gives up in exchange: a bounded vocabulary, an upfront semantic layer construction cost, and dependence on intent-extraction quality for semantic (though never safety) correctness. Within that scope, this thesis demonstrates that

restricting SQL generation to a finite set of validated business patterns is a practical, measurable path to safe, auditable natural-language reporting in institutional environments.

REFERENCES

[1] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911-3921.

[2] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[3] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," The VLDB Journal, vol. 28, no. 5, pp. 793-819, 2019.

[4] M. Liu and J. Xu, "NLI4DB: A systematic review of natural language interfaces for databases," arXiv:2503.02435, 2025.

[5] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," Proceedings of the VLDB Endowment, vol. 8, no. 1, pp. 73-84, 2014.

[6] C. Lehmann, D. Gehrig, S. Holdener, C. Saladin, J. P. Monteiro, and K. Stockinger, "Building natural language interfaces for databases in practice," in Proc. 34th Int. Conf. Scientific and Statistical Database Management (SSDBM), 2022, Art. no. 20.

[7] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7567-7578.

[8] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 9895-9901.

[9] H. S. Shalaan, T. H. A. Soliman, and A. M. Abdelaziz, "G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation," IEEE Access, vol. 13, pp. 158520-158534, 2025.

[10] X. Su, Y. Gu, P. Wang, W. Gu, L. Qi, and J. He, "A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs," Scientific Reports, vol. 16, Art. no. 7892, 2026.

[11] A. Narechania, A. Srinivasan, and J. Stasko, "nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

[12] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in Proc. 28th Annual ACM Symposium on User Interface Software and Technology (UIST), 2015, pp. 489-500.

[13] D. Deng, A. Wu, H. Qu, and Y. Wu, "DashBot: Insight-driven dashboard generation based on deep reinforcement learning," IEEE Transactions on Visualization and Computer Graphics, vol. 29, no. 1, pp. 690-700, 2023.

[14] G. N. Shailesh, M. Pavithran, A. Rahul Hari Venkat, and P. Kaliappan, "Conversational BI: Natural language interface to business dashboards," International Journal of Engineering Research & Technology, vol. 14, no. 12, 2025.

[15] J. Valkenburgh, "Enhancing business dashboards with explanatory analytics and AI: Exploring the use of AI and explanatory analytics to enhance business decision-making," M.S. thesis, Information Management, Turku School of Economics, University of Turku, Finland, 2024.

[16] OpenAI, "Introducing structured outputs in the API," OpenAI, 2024.